B.Tech Computer Engineering - 2nd Year (3rd Semester)

Data Structure Lab CSL-302 | Lab Assignment | 27 October 2026

Implementation of a Double Ended Queue Using an Array in C

Student: YOUR NAME | Roll number: 26BCS001

Objective

Implement a menu-driven deque using a circular array. It supports insertion and deletion at either end, reading front/rear values, display, empty/full checks, and exit. The source uses capacity 5; change CAPACITY if your lab requires a different size.

Representation and circular indexing

The array deque stores values. front and rear identify the first and last occupied slots. Both are -1 when empty. Circular movement uses modulo CAPACITY so slots freed at the beginning can be reused. The deque is full when (front == 0 and rear == CAPACITY - 1) or (front == rear + 1).

IsEmpty()

Parameters: none. Return value: bool. Checks whether front is -1, the representation of an empty deque. Called before deletion, access, and display.

IsFull()

Parameters: none. Return value: bool. Checks the two circular-array full conditions so insertion never overwrites a stored element.

Insertfront()

Parameters: none; reads the value from standard input. Return value: none. It rejects a full deque, initializes both indices when empty, otherwise moves front one position backward with wrap-around, then stores the new value.

InsertRear()

Parameters: none; reads the value from standard input. Return value: none. It rejects a full deque, initializes an empty deque, otherwise advances rear with wrap-around, then stores the value.

Deletefront()

Parameters: none. Return value: none. It reports an empty deque; otherwise prints the removed front value. If it was the last item both indices reset to -1; otherwise front advances circularly.

DeleteRear()

Parameters: none. Return value: none. It reports an empty deque; otherwise prints the removed rear value. It resets both indices for the last item or moves rear backward circularly.

getRear() and getFront()

Parameters: none. Return value: none; each prints the requested value or an empty message. They inspect deque[rear] or deque[front] without changing the deque.

display()

Parameters: none. Return value: none. Starting at front, it prints values while advancing modulo CAPACITY until rear is reached. This visits exactly the occupied elements, including across the array boundary.

main() and menu flow

Parameters: none. Return value: 0 on normal exit. Prints the roll number/name, repeatedly displays the ten menu choices, reads a choice, and calls the matching function. Invalid choices are reported; choosing 10 ends the loop. Every function has a comment describing its purpose, parameters, return value, and key logic, and exactly one printf statement for roll number/name as required. Other output uses puts, putchar, or value-printing statements.

Complexity and checks

Insert, delete, front/rear access and empty/full checks take O(1) time. Display takes O(n), where n is the current number of items. Storage is O(CAPACITY). Try insertion at both ends, wrap-around, deletion of the final item, empty/full operations, displaying a wrapped deque, invalid menu input, and Exit.

Before submission

Replace YOUR NAME in the C source and this report with your own name. Confirm the roll number is correct and review the report against your final code so you can explain it in the viva.